

Clerk: Real Auth Without Writing Auth

Sign Up, Sign In, Per-User Data — In One Evening, on the Free Tier

CSCI 39548 — Practical Web Development

Bonus Segment

Where We Are Tonight

By now you have:

- ▶ A React + Vite frontend
- ▶ An Express + Prisma backend
- ▶ A Neon Postgres database
- ▶ A public URL (from the deploy block)

There is one thing everyone on the internet can now do to your app:

Open it, add todos, delete other people's todos, and nothing stops them.

Tonight: add real accounts.

What “Auth” Actually Means

Two words that get confused constantly:

- ▶ **Authentication** — *who are you?* (login: email + password, or Google, or a passkey)
- ▶ **Authorization** — *are you allowed to do this?* (only the owner can delete their own todo)

Clerk handles the first one for us. We wire the second one ourselves.

Everything else in this deck is one of those two words.

Why “Just Write It Yourself” Is a Trap

To do auth properly you need:

- ▶ Password hashing (bcrypt / argon2 — get the parameters wrong, you leak passwords)
- ▶ Email verification (send + expire + validate tokens)
- ▶ Password reset flow (send + expire + validate again)
- ▶ Session storage (cookies? tokens? both? how long?)
- ▶ Rate limiting (or brute-force wins)
- ▶ 2FA / passkeys (users expect this now)
- ▶ Social login (OAuth dance with Google/GitHub/Apple)
- ▶ Account recovery, account deletion, GDPR export...

Every one of these is a way to get pwned. Auth is the #1 thing student projects get wrong.

The Managed-Auth Pitch

Auth is a **solved problem**. Multiple companies sell it as a service:

Service	What they give you
Clerk	Prebuilt UI + hosted user DB + React/Express SDKs
Auth0	The original enterprise auth-as-a-service
Supabase Auth	Auth bundled with Postgres-as-a-service
Firebase Auth	Google's version, tied to Firebase

We use **Clerk**: prebuilt UI (no login form to design), free tier fits students, React SDK is dead simple, and the dashboard is great UX.

The 3-Piece Flow

Every managed-auth setup has three pieces:

1. **Clerk** — hosts the login UI, stores users, issues signed tokens
2. **Your frontend** — asks Clerk “who is this?”, attaches the token to every API call
3. **Your backend** — verifies the token, uses the user id to scope data

```
[User] -> [Clerk UI] -> (signed token) -> [React]
      -> (Bearer token) -> [Express] -> [Postgres]
```

Nothing else. That's the entire architecture.

What Is a JWT (60 Seconds)

Clerk gives your frontend a **JWT** — a JSON Web Token:

eyJhbGciOi...	.	eyJzdWIiOi...	.	abc123signature
header		payload		signature

Three parts, base64-encoded, separated by dots.

- ▶ **Payload** is readable — contains sub (user id), exp (expiry), etc.
- ▶ **Signature** is unforgeable — signed with Clerk's private key. Your backend verifies it with Clerk's public key.

A JWT is a note from Clerk that says *"this user is real, I checked, and here is their id."* Your backend trusts the note because it's signed.

Per-User Scoping (The Actual Point)

Before Clerk:

```
// OPEN -- anyone sees everything
const todos = await prisma.todo.findMany();
```

After Clerk:

```
// SCOPED -- you see only yours
const todos = await prisma.todo.findMany({
  where: { userId: req.auth.userId },
});
```

This one-line change is the whole payoff.

Clerk's Job vs Your Job

Clerk's job	Your job
Store users (email, password hash, profile)	Store <i>their data</i> (todos, posts, etc.)
Render the sign-in / sign-up UI	Show your app once they're signed in
Issue JWTs	Verify JWTs on every backend request
Handle password reset, email verification	Attach <code>userId</code> to every row you write
Manage sessions	Filter every query by <code>userId</code>

You never write a login form. You never store a password. You never send a verification email.

Prerequisites

Before we start:

- ▶ Working Vite + React + TypeScript frontend
- ▶ Working Express + Prisma + Neon backend
- ▶ Both boot cleanly with `npm run dev`
- ▶ You've signed up for exactly zero auth services

Tonight we add: a Clerk account, one env var on each side, a schema column, and about 20 lines of code.

Create a Clerk Account

Go to **clerk.com** → *Sign up*. Free tier: 10,000 monthly active users. You will not hit this limit.

Once signed in: **Create application**. Name it (e.g. csci39548-demo).



csci39548-demo

Configure

 User & authentication

 Sessions

 Organizations

 Restrictions

 API keys

 Users

 Updates

 Sourya Saha

sourya.saha@example.com

csci39548-demo

Hobby

Welcome to Clerk! Here's what you can do next.

Quick start

Set up your application with Clerk in just a few steps.



Add sign in

Add secure sign in and sign up to your application.

Get started →



Add user profile

Collect and manage additional user information.

Get started →



Add multi-factor

Enable 2FA to keep your users' accounts secure.

Get started →

 Documentation

Learn how to integrate Clerk in your app.

View docs

CSCI 39548

12

Pick Your Sign-In Methods

Sidebar → **Configure** → **User & authentication** → **Email, Phone, Username.**

- ▶ ✓ Email address (required)
- ▶ ✓ Password
- ▶ × Phone (skip — costs money at scale)
- ▶ × Username (skip — email is enough)

You can also toggle social login (Google, GitHub) here. Optional — email + password is what we walk in class.

User & Authentication Config

The screenshot displays the Clerk dashboard's 'User & authentication' configuration page. On the left is a dark sidebar with the Clerk logo, a demo account selector ('csci39548-demo'), and a 'Configure' menu with options for User & authentication, Sessions, Organizations, Restrictions, API keys, Users, and Updates. The main content area is titled 'User & authentication' and includes a breadcrumb 'Configure > User & authentication', environment and user selectors ('Development' and 'SS'), and a 'Save' button. The configuration is divided into two sections: 'Contact information' and 'Authentication strategies'. The 'Contact information' section contains three items: 'Email address' (enabled, with 'Email verification code' selected as the verification method), 'Phone number' (disabled), and 'Username' (disabled). The 'Authentication strategies' section contains 'Password' (enabled, with 'Medium' strength requirements selected).

clerk

csci39548-demo

Configure

- User & authentication
- Sessions
- Organizations
- Restrictions
- API keys
- Users
- Updates

Sourya Saha
surya.saha@example.com

Configure > User & authentication

Development SS

User & authentication

Choose how users sign in to your application.

Save

Contact information

Email address
Allow users to sign in with their email address. ☒

Verification method

- ☒ Email verification code
- ☐ Email verification link
- ☐ Password reset code

Phone number
Send SMS codes for sign-in and verification. ☐

Username
Let users pick a unique username to sign in. ☐

Authentication strategies

Password
Allow users to sign in with a password. ☒

Strength requirements

Medium

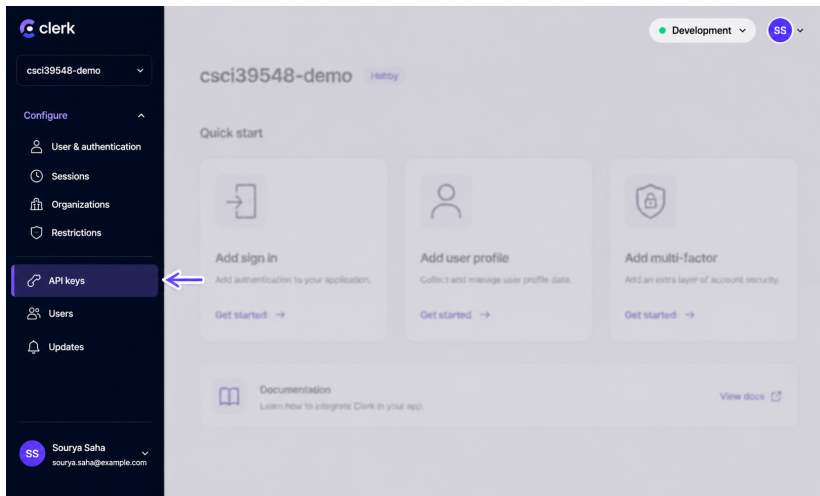
Find Your API Keys

Sidebar → **API keys**. Every Clerk app has two keys:

- ▶ **Publishable key** — starts with `pk_test_...` Safe to expose in the browser.
- ▶ **Secret key** — starts with `sk_test_...` **Backend only**. Never ship this to the browser.

Copy both. We'll put one on the frontend and one on the backend.

API Keys — Where to Click



The API Keys Page

Two boxes, each with a Copy button:

- ▶ **Publishable key** — long visible string starting `pk_test_ZXF1...`
- ▶ **Secret key** — masked by default (`sk_test_●●●●`), click eye icon to reveal

The publishable key also encodes your app's Frontend API URL. Clerk's SDK reads this from the key itself.

API Keys Page

The screenshot shows the Clerk dashboard's 'API keys' section. On the left is a dark sidebar with the Clerk logo, a user dropdown (csci39548-demo), and a 'Configure' menu with options: User & authentication, Sessions, Organizations, Restrictions, API keys (selected), Users, and Updates. At the bottom of the sidebar is the user profile for Sourya Saha. The main content area is titled 'API keys' and includes a 'Development' environment selector. A warning message states: 'These keys are for your development instance. Switch to production later.' Below this are tabs for 'Next.js', 'React' (active), and 'Node.js'. The 'Publishable key' is shown as 'pk_test_ZXF1aXBwZWQtc2F3Zm1zaC00S5jbGVyay5hY2NvdW50ccy5kZXYk' with a 'Copy' button. The 'Secret key' is shown as 'sk_test_...' with a 'Copy' button and a warning 'Backend only'.

clerk

csci39548-demo

Configure

User & authentication

Sessions

Organizations

Restrictions

API keys

Users

Updates

Sourya Saha

sourya.saha@example.com

API keys

Development

SS

API keys

These keys identify your application to Clerk. Rotate them if compromised.

ⓘ

These keys are for your development instance. Switch to production later.

Next.js

React

Node.js

Publishable key

Safe to expose in your frontend bundle.

pk_test_ZXF1aXBwZWQtc2F3Zm1zaC00S5jbGVyay5hY2NvdW50ccy5kZXYk

Copy

Secret key

⚠ Backend only

Grants full API access. Never ship this to a browser.

sk_test_.....

Copy

Why TWO Keys?

Because the browser cannot be trusted with the secret.

- ▶ The **publishable key** goes into your frontend bundle. Users can view it in DevTools. That is **fine** — it identifies your Clerk app but grants no privileges beyond “show me a login form.”
- ▶ The **secret key** proves that a request is from *your backend* to Clerk. If leaked, an attacker could impersonate the backend.

The frontend key is a business card. The secret key is a signed cheque.

Where Each Key Goes

frontend/.env	backend/.env
-----	-----
VITE_CLERK_PUBLISHABLE_KEY	CLERK_SECRET_KEY
= pk_test_...	= sk_test_...
	CLERK_PUBLISHABLE_KEY
	= pk_test_...

- ▶ Frontend: VITE_ prefix required (Vite only exposes VITE_* vars)
- ▶ Backend: no prefix — Express reads via `process.env.CLERK_SECRET_KEY` + `process.env.CLERK_PUBLISHABLE_KEY`
- ▶ The **publishable key is the same value on both sides** — copied twice, that's fine. It identifies the app; both sides need to know which Clerk app they're talking to.
- ▶ Both `.env` files are in `.gitignore`. Do not commit them.

Install the SDKs

Two packages. Two `npm install` commands.

Frontend (in the Vite project):

```
npm install @clerk/clerk-react
```

Backend (in the Express project):

```
npm install @clerk/express
```

No auth logic to write — just wire up the components.

Set Up the Frontend .env

```
VITE_CLERK_PUBLISHABLE_KEY=pk_test_ZXF1aXBwZWQt...  
VITE_API_URL=http://localhost:3000
```

- ▶ Vite only exposes vars starting with `VITE_` to `import.meta.env`
- ▶ The SDK reads the publishable key from `import.meta.env.VITE_CLERK_PUBLISHABLE_KEY`
- ▶ Restart `npm run dev` after editing `.env` — Vite reads it at startup, not per-request

Set Up the Backend .env

```
DATABASE_URL=postgresql://...neon.tech/neondb?sslmode=require
CLERK_SECRET_KEY=sk_test_...
CLERK_PUBLISHABLE_KEY=pk_test_...
FRONTEND_ORIGIN=http://localhost:5173
PORT=3000
```

- ▶ DATABASE_URL — Neon branch for this demo
- ▶ CLERK_SECRET_KEY — from the dashboard
- ▶ CLERK_PUBLISHABLE_KEY — **yes, the backend needs it too** (same value as frontend). @clerk/express v1.3+ verifies the JWT's iss claim against it.
- ▶ FRONTEND_ORIGIN — CORS whitelist (Vite runs on 5173)
- ▶ PORT — Express listens here in dev

Create a Neon Branch for This Demo

We don't want auth experiments messing up our deploy demo's data. Neon **branches** solve this:

- ▶ Go to your Neon project → *Branches* → **Create branch** → name it clerk-demo
- ▶ Copy the pooled connection string → paste into backend/.env as DATABASE_URL

Branches are copy-on-write forks of your database. Free, instant, isolated. Use one per experiment.

Neon Branches

NEON

PROJECTS

todo-app

Branches

Tables

SQL Editor

Roles & Permissions

Settings

SS

Sourya Saha

Free plan

todo-app > Branches

Branches

+ Create branch

Isolated copies of your database. Free, instant, copy-on-write.

Name	Parent	Created	Compute	Status	
main	—	3 days ago	0.25 CU	Ready	...
clerk-demo	main	just now	0.25 CU	Ready	...

Connection details

Pooled connection

Direct connection

postgresql://neondb_owner:npg_*****@ep-aged-union-aq8z10r8-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require

Copy

Pooled endpoint — use this for Vercel, Render, or any serverless. Contains -pooler in the hostname.

Frontend Changes: Overview

Three edits, all in the Vite project:

1. `main.tsx` — wrap `<App />` in `<ClerkProvider>`
2. `App.tsx` — gate with `<SignedIn>` / `<SignedOut>`
3. **New file:** `SignIn.tsx` — one-liner using Clerk's `<SignIn />` component

That's the entire frontend integration.

```
import { ClerkProvider } from "@clerk/clerk-react";

const PUBLISHABLE_KEY =
  import.meta.env.VITE_CLERK_PUBLISHABLE_KEY;
if (!PUBLISHABLE_KEY) {
  throw new Error("Missing VITE_CLERK_PUBLISHABLE_KEY");
}

createRoot(document.getElementById("root")!).render(
  <ClerkProvider publishableKey={PUBLISHABLE_KEY}>
    <App />
  </ClerkProvider>
);
```

Why the Fail-Fast Check?

That `throw new Error("Missing VITE_CLERK_PUBLISHABLE_KEY")` is deliberate.

- ▶ Without it, `ClerkProvider` renders silently with `publishableKey={undefined}`
- ▶ Nothing crashes. Nothing signs in. You spend an hour wondering why.
- ▶ **With the throw**, the app refuses to boot until the env var is set

Fail loud. Fail early. Fail with a message that tells you exactly what to fix.

Gate the App in App.tsx

```
import { SignedIn, SignedOut, UserButton }
  from "@clerk/clerk-react";
import SignInPage from "../SignIn";

export default function App() {
  return (
    <
      <SignedOut>
        <SignInPage />
      </SignedOut>
      <SignedIn>
        <header><UserButton /></header>
        <TodoApp />
      </SignedIn>
    </>
  );
}
```

What <SignIn /> Renders

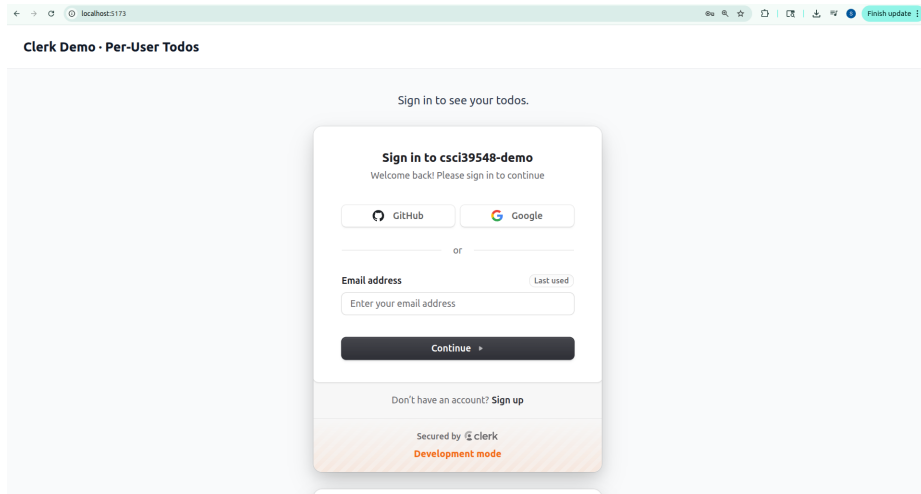
```
// SignIn.tsx
import { SignIn } from "@clerk/clerk-react";

export default function SignInPage() {
  return <SignIn />;
}
```

You get: email input, password input, “forgot password” link, “sign up instead” link, error handling, email verification flow — all of it.

This is why we picked Clerk. That one-liner is a login form that would take you a week to build correctly.

The <SignIn /> Component



What `<UserButton />` Gives You

One component. Renders as the user's avatar in the top-right.

Click it → dropdown with:

- ▶ Manage account (opens Clerk's profile UI — change email, password, add 2FA)
- ▶ Sign out

You never write any of this. You never build a profile page. You never write a sign-out button.

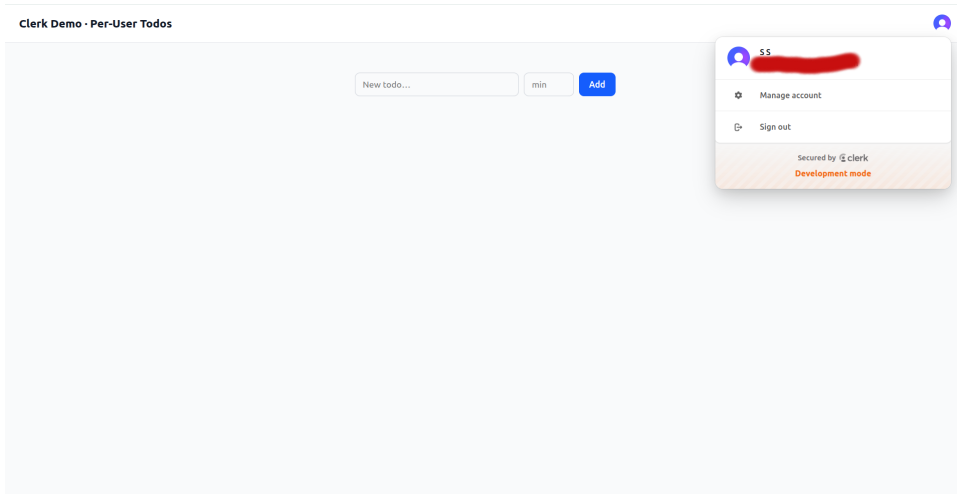
First Boot Test

Save all three files, boot the frontend (`npm run dev`), open `http://localhost:5173`.

Expected: Clerk sign-in card, nothing else.

Sign up with a real email → get a verification code → paste it → land on your todo app with the `UserButton` in the top-right.

After Sign-In



The Cliffhanger

You are signed in. You add a few todos. They persist.

Now sign out. Sign back in as a **different** user.

You still see the same todos.

- ▶ The frontend gates *what UI* you see
- ▶ It does **not** gate *what data* the backend sends you
- ▶ The backend is still returning every row

This is what the backend changes fix.

Backend Changes: Overview

Three edits, all in the Express project:

1. `server.ts` — mount `clerkMiddleware()` so every request has `req.auth`
2. `routes/todos.ts` — read `req.auth.userId` and scope every query
3. **Frontend** `App.tsx` — attach the JWT to every fetch as `Authorization: Bearer`

Add the Middleware

```
// server.ts
import { clerkMiddleware } from "@clerk/express";

app.use(cors({
  origin: process.env.FRONTEND_ORIGIN,
  credentials: true,
}));
app.use(express.json());
app.use(clerkMiddleware());           // <-- new
app.use("/api/todos", todosRouter);
```

Order matters. `clerkMiddleware()` must run *before* your routes, so `req.auth` exists by the time handlers read it.

What `clerkMiddleware()` Actually Does

On every request:

1. Read the Authorization: Bearer <jwt> header
2. If no header $\rightarrow$ `req.auth = { userId: null }`
3. If header present $\rightarrow$ verify the JWT's signature using Clerk's public key
4. If signature valid $\rightarrow$ `req.auth = { userId: "user_abc123", ... }`
5. If signature invalid $\rightarrow$ same as no header (`userId: null`)

It does **not** reject unauthenticated requests. It just tells your handler *who* the request is from. Your handler decides whether to allow it.

Scope Every Query by userId

```
router.get("/", asyncHandler(async (req, res) => {
  const { userId } = req.auth;
  if (!userId) return res.status(401).json({ error: "Unauthorized"
    });

  const todos = await prisma.todo.findMany({
    where: { userId },           // <-- the one-line payoff
    orderBy: { id: "asc" },
  });
  res.json(todos);
}));
```

401 if no userId. where: { userId } — Prisma filters at the SQL layer; users literally cannot see each other's rows.

POST / PATCH / DELETE: Ownership Checks

```
// POST: stamp the row with the caller's userId
const todo = await prisma.todo.create({
  data: { ...result.data, userId },
});

// PATCH / DELETE: verify ownership before mutating
const existing = await prisma.todo.findUnique({
  where: { id },
});
if (!existing || existing.userId !== userId) {
  return res.status(404).json({ error: "Not found" });
}
```

We return **404**, not **403**, for “not yours.” This hides the existence of other users’ rows.

Frontend: Attach the JWT

```
import { useAuth } from "@clerk/clerk-react";

function TodoApp() {
  const { getToken } = useAuth();

  async function loadTodos() {
    const token = await getToken();
    const res = await fetch(`${API}/api/todos`, {
      headers: { Authorization: `Bearer ${token}` },
    });
    return res.json();
  }
}
```

Same on POST/PATCH/DELETE. Every request. Every time.

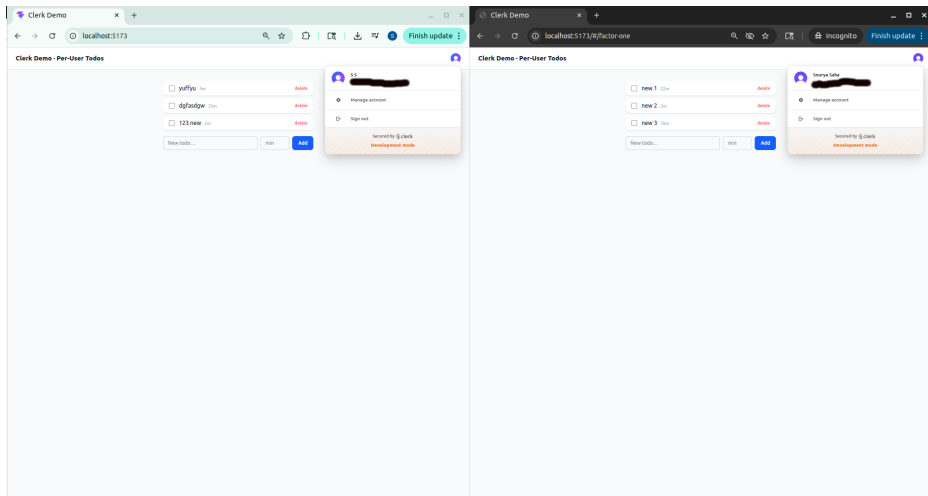
End-to-End Smoke Test

Two browser windows side-by-side (normal + incognito). Sign in as different users.

- ▶ **Window A (user X):** add “Read chapter 3” → visible in A
- ▶ **Window B (user Y):** add “Buy milk” → visible in B
- ▶ Window A shows only X’s todos. Window B shows only Y’s.
- ▶ Delete a todo in A → gone in A, never existed in B

This is the moment auth is real. Two users, one database, complete isolation.

Two-Window Isolation



Neon Cross-Check

Open Neon's **Tables** view for the clerk-demo branch → Todo table.

Every row now has a real `userId` like `user_2abc...`. Rows from user X and user Y are mixed in the same table but distinguishable by the `userId` column.

Nothing in the schema is “auth-aware.” The isolation is entirely enforced by `WHERE userId = ...` in your Prisma queries.

Neon Tables — Per-User Rows

The image displays two side-by-side screenshots of the Neon Console interface, showing the 'Tables' view for a project named 'fancy-bonus-0827730'. The interface is divided into a sidebar, a top bar, and a main content area.

Left Screenshot:

- Top Bar:** Shows the project name 'fancy-bonus-0827730' and a 'Finish update' button.
- Sidebar:** Contains navigation links for Dashboard, Branches, Integrations, Settings, Overview, Monitoring, SQL Editor, Tables, Backup & Restore, Data Masking, Data API, and Auth.
- Main Content Area:** Displays the 'Tables' view for the 'records' table. The table has 6 rows and 5 columns: id, email, name, date_created, and minutes_created. The 'id' column is highlighted with a filter 'id = user_3f67NvMf37nDg'. The 'email' column is highlighted with a filter 'email = user_3f67NvMf37nDg'.

Right Screenshot:

- Top Bar:** Shows the project name 'fancy-bonus-0827730' and a 'Finish update' button.
- Sidebar:** Contains navigation links for Dashboard, Branches, Integrations, Settings, Overview, Monitoring, SQL Editor, Tables, Backup & Restore, Data Masking, Data API, and Auth.
- Main Content Area:** Displays the 'Tables' view for the 'records' table. The table has 6 rows and 5 columns: id, email, name, date_created, and minutes_created. The 'id' column is highlighted with a filter 'id = user_3f67NvMf37nDg'. The 'email' column is highlighted with a filter 'email = user_3f67NvMf37nDg'.

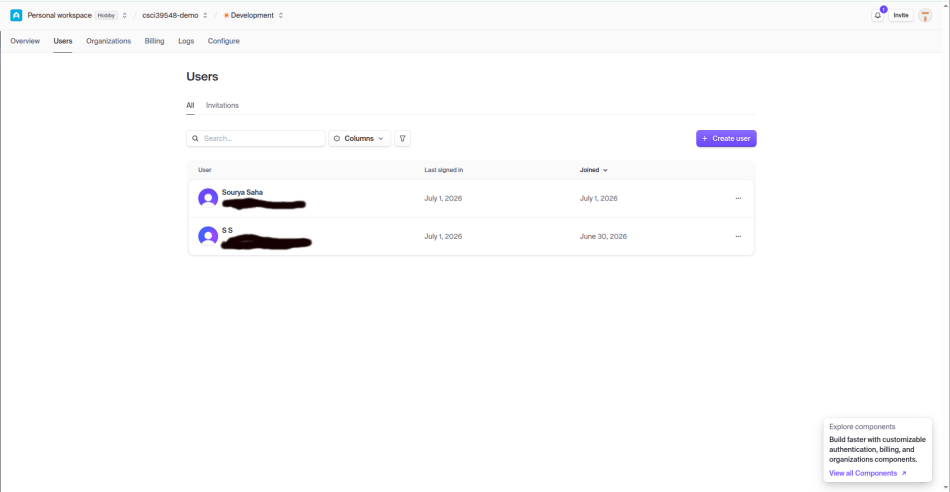
Clerk Dashboard: Users View

Sidebar → **Users**. Every user who has signed up is here.

- ▶ Search by email
- ▶ Ban / delete accounts
- ▶ View sign-in history
- ▶ Manually reset passwords
- ▶ Impersonate users (for support)

You never built a user table. You never wrote a user-management page. Clerk gave you one.

Clerk Users Page



Common Errors (1 of 2)

Error	Cause	Fix
Missing publishable key on boot	.env not read	Restart npm run dev
publishableKey is required	Prefix wrong (no VITE_)	Rename to VITE_CLERK_PUBLISHABLE_KEY
Publishable key missing in backend logs	Backend .env missing CLERK_PUBLISHABLE_KEY	Add it (same value as frontend), restart
Sign-up rejected	Email/password not enabled	Configure → User & auth
401 on every API call	Bearer token not attached	Add Authorization header

Common Errors (2 of 2)

Error	Cause	Fix
CORS error with credentials	Backend uses wildcard origin	Set <code>FRONTEND_ORIGIN</code> explicitly
<code>req.auth</code> is undefined	Middleware not mounted or after routes	Mount <code>before</code> <code>app.use("/api/...")</code>
Signed in but 401	<code>getToken()</code> returned null	Sign out and back in
Todos still shared	Forgot <code>where: { userId }</code>	Add the filter, restart

CORS with Credentials

Once you attach an Authorization header, browsers treat the request as **credentialed**:

```
app.use(cors({
  origin: process.env.FRONTEND_ORIGIN, // NOT "*"
  credentials: true,
}));
```

- ▶ Access-Control-Allow-Origin must be a **specific origin** — "*" is forbidden with credentials
- ▶ Response must include Access-Control-Allow-Credentials: true

This is why we stopped using origin: "*" in the deploy block. It was preparing you for this.

Env Vars Master Reference

Where	Variable	Value	Why
Frontend .env	VITE_CLERK_PUBLISHABLE_KEY	pk_test_...	Clerk React SDK
Frontend .env	VITE_API_URL	http://localhost:3000	Where fetches go
Backend .env	CLERK_SECRET_KEY	sk_test_...	Verifies JWTs
Backend .env	CLERK_PUBLISHABLE_KEY	pk_test_...	SDK v1.3+ requires it
Backend .env	DATABASE_URL	Neon pooled URL	Prisma connects
Backend .env	FRONTEND_ORIGIN	http://localhost:5173	CORS whitelist
Backend .env	PORT	3000	Dev only

Screenshot this slide.

What You Have Now

- ▶ Real user accounts with email verification
- ▶ A hosted sign-in / sign-up UI you didn't write
- ▶ Per-user data isolation enforced at the database query layer
- ▶ A user management dashboard for free
- ▶ Sign-out and account management for free
- ▶ ~30 lines of code across frontend + backend

This would be a week of work without Clerk. It was one evening.

Two-Window Test (Recap)

The single most important test you can run:

1. Open your app in a normal window and an incognito window
2. Sign in as different users in each
3. Add distinct todos in each
4. Confirm: neither window can see the other's data
5. Delete a todo in one — confirm it's gone there and never appeared elsewhere

If both windows show the same data, your backend isn't scoping by `userId`.

When NOT to Use Clerk

- ▶ **Lock-in** — migrating away means re-registering every user
- ▶ **PII on Clerk's servers** — medical/financial apps with strict compliance may be disqualified
- ▶ **Rate limits + cost** — 10k MAUs free, past that you pay
- ▶ **Offline apps** — Clerk needs to reach `clerk.accounts.dev`

For a course project, a startup MVP, or an internal tool: **use Clerk**. For a bank: talk to a security architect.

Deploying with Clerk

Same three services (Vercel + Render + Neon), plus two extra env vars:

- ▶ Vercel → add `VITE_CLERK_PUBLISHABLE_KEY` and `VITE_API_URL`
- ▶ Render → add `CLERK_SECRET_KEY`, `CLERK_PUBLISHABLE_KEY`, and `FRONTEND_ORIGIN` (Vercel URL, no trailing slash)

Redeploy both. Public users sign up on your public URL.

The auth wiring does not change between dev and prod. Only the env vars do.

Dev Instance vs Production Instance

- ▶ `pk_test_` and `sk_test_` keys are for a **development instance**
- ▶ Clerk gives a separate **production instance** when you're ready to ship
- ▶ Dashboard: top-left → switch instance → *Production*
- ▶ You get `pk_live_` and `sk_live_` — dev users don't carry over (intentional)

For the course, `_test_` keys are fine everywhere. Switch to `_live_` when the app has real users.

For Your Final Project

If your app has anything user-owned, you need auth. Options:

1. **Use Clerk.** Everything in this deck applies. Free tier covers the class.
2. **Use Supabase Auth** if you're already using Supabase for your DB.
3. **Don't use Firebase Auth** unless you're deep in Firebase.
4. **Do not roll your own.**

“How is auth handled” is the first question your grader will ask about a fullstack project.

Gotcha Summary

- ▶ **Restart the dev server** after editing `.env`
- ▶ `VITE_` **prefix** or the browser can't see the var
- ▶ **Middleware order matters** — `clerkMiddleware()` before routes
- ▶ `credentials:` `true` in CORS + a specific origin (never `"*"`)
- ▶ **404 not 403** for “not yours” — hide row existence
- ▶ **Attach the token on every fetch** — not just on load
- ▶ **Neon branches** for auth experiments — don't pollute deploy DB

What You Didn't Build Tonight

Things Clerk gave you for free that would otherwise be days of work:

- ▶ Password hashing (bcrypt/argon2)
- ▶ Email verification tokens
- ▶ Password reset flow
- ▶ Session refresh + expiry
- ▶ Rate limiting on login endpoints
- ▶ CSRF protection on session cookies
- ▶ A user profile UI
- ▶ A “manage sessions” screen
- ▶ Multi-factor authentication support
- ▶ Google / GitHub OAuth (one toggle to add)

Every one of these has bit real companies.

What Next

If you want to go further:

- ▶ **Roles + permissions** — Clerk supports organizations (teams, admins, members)
- ▶ **Webhooks** — Clerk POSTs to your backend when a user signs up
- ▶ **Custom claims** — attach app-specific data to the JWT
- ▶ **Passkeys** — passwordless login, one dashboard toggle

None of this is on the exit. All of it is fair game for your final project.

Recap

Tonight, in order:

1. Signed up for Clerk, got two keys
2. Added a Neon branch for isolation
3. Wrapped React in `<ClerkProvider>`, gated with `<SignedIn>/<SignedOut>`
4. Added `<SignIn />` and `<UserButton />` — never wrote a form
5. Mounted `clerkMiddleware()` on Express, read `req.auth.userId`
6. Filtered every Prisma query by `userId`
7. Attached Authorization: Bearer to every fetch
8. Verified with a two-window test

Real auth. Real per-user data. One evening.

Going Deeper

- ▶ **Clerk docs** — `clerk.com/docs` — start with React + Express quickstarts
- ▶ **JWT explainer** — `jwt.io` — paste any token, see the payload decoded
- ▶ **OWASP Auth Cheat Sheet** — what Clerk actually protects you from
- ▶ **Auth0 blog** — competitor, but their OAuth flow explainer posts are excellent

The one thing you should *not* do is write your own password hashing.